Lab 4: Hardware Data Prefetcher Design and Analysis

Introduction

In this lab, I implement a GHB-based stride hardware data prefetcher. In a later step I improve the design by making it aware of system memory bandwidth constraints. Finally I create my own hybrid prefetcher, which chooses between a best-offset-prefetcher (Michaud 2016) and the previous strided GHB prefetcher.

Warm Up: Running the No-prefetching Baseline

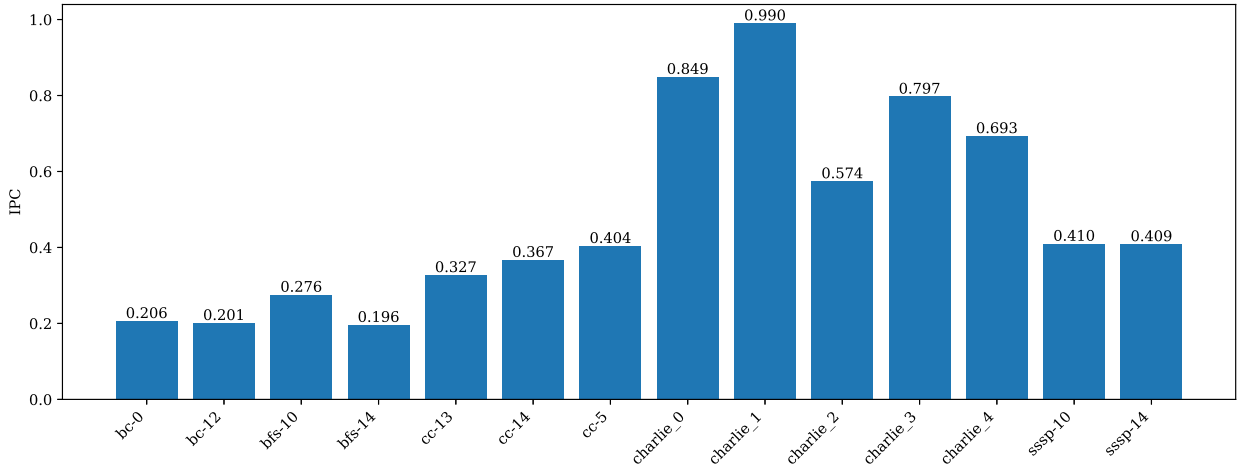

Figure 1: Baseline performance of the system without an L2C prefetcher on real-world CPU traces

The hardware prefetcher designs are evaluated on CPU traces collected from fourteen workloads sourced from graph analytics benchmarks and data-center workloads. Each workload was run for 50 million cycles following 10 million cycles of initial warmup using the Champsim microarchitecural simulator (Gober et al. 2022). The simulated system has a single out-of-order core and a single channel of DRAM running at a data rate of 4800 MT/s ("full BW").

In fig. 1 is a plot of the instructions-per-cycle (IPC) for each workload. The `charlie_1` trace resulted in the highest IPC value, while the `bfs-14` trace got the lowest IPC value.

Task 1/4: Implementing GHB-based Stride Prefetcher

Implementation

In Task 1 I implemented a GHB-based stride prefetcher (Nesbit and Smith 2004) for the L2C as closely as possible to the original specification. I took special care to accurately replicate the bit-width of tags in the Index Table (IT) and pointers in the Global History Buffer (GHB). However in my own testing, I found that the IPC results for the exact replication are identical to those with a more relaxed replication of the paper with wider bit-widths (tbl. 1). First, this shows that the hardware budget proposed in the paper provides performance equal to an expensive prefetcher, but with a fraction of the resources. Second, this means that my replication of the original prefetcher isn't sensitive to the specific sizings of the table entries.

Table 1: Per-trace IPC for GHB PC/CS with 20b and 56b wide IT tags (1 cpu, full BW)

trace	1	2	3	4	5	6	7	8	9	10	11	12	13	14
20b IPC	0.229	0.221	0.362	0.293	0.415	0.476	0.532	0.864	1.014	0.582	0.809	0.704	0.548	0.554
56b IPC	0.229	0.221	0.362	0.293	0.415	0.476	0.532	0.864	1.014	0.582	0.809	0.704	0.548	0.554

The per-workload IPC speedups, prefetching accuracies and prefetches per thousand instructions (PPKI) caused by the strided GHB prefetcher in the L2C are plotted in fig. 2. The IPC speedup is simply the ratio of the IPC measured with the prefetcher and the IPC without the prefetcher. Next, the accuracy is defined as the ratio of

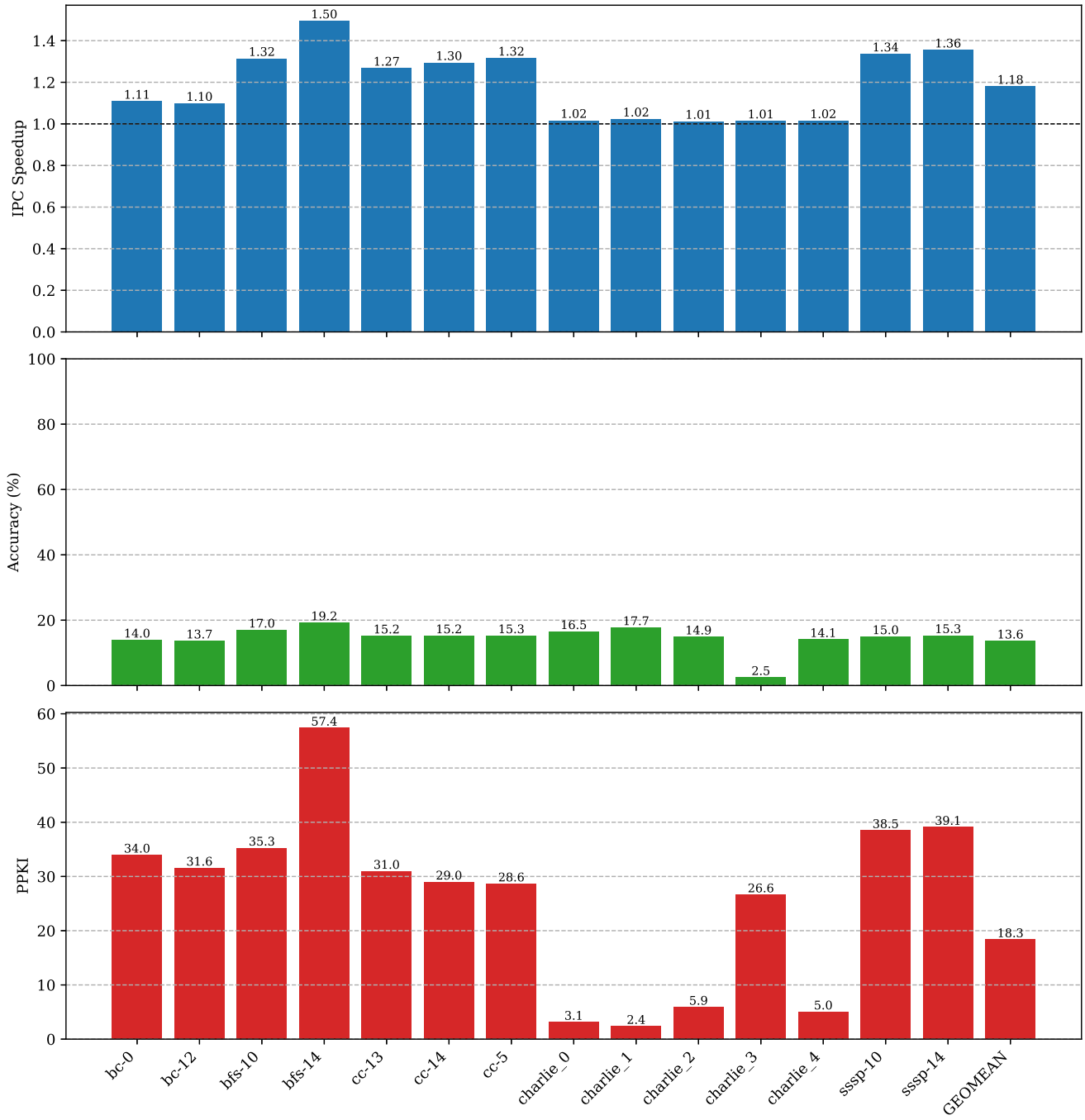

Figure 2: Task 1: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher (1 CPU, full BW)

prefetches reported by Champsim logs as being "useful" (`l2c_prefetch_useful`) and the total number of issued prefetches (`l2c_prefetch_issued`).

Results

The utility of the prefetcher is highly heterogenous for the different types of workloads. As explained in the following paragraphs, this indicates that the strided GHB prefetcher is only useful for workloads with access patterns that play to its strengths. Notice that the IPC speedup is always greater than 1, implying that the prefetcher gains a strict performance increase.

GAP Workloads `bfs-14` was the workload with the largest IPC speedup. It was also the workload with the largest amount of prefetches issued, at the highest accuracy. The large number of prefetches issued indicate that the GAP workloads (graph analytics) can benefit a lot from a strided GHB prefetcher, considering they are most likely traversing neighboring nodes lists.

Charlie Workloads `charlie_2` got the lowest IPC speedup of all workloads. With the exception of `charlie_3`, very few prefetches were issued for this group of workloads, leading to very low IPC speedups overall. For `charlie_3`, the number of issued prefetches was much higher, but with very poor accuracy. This indicates that the charlie workloads (sourced from Google data centers) feature cache accesses that don't suit a constant-stride prefetcher's profile. Possibly these workloads resemble pointer-chasing or random accesses.

Task 2/4: Prefetching with Limited Main Memory Bandwidth

In this task, I investigate the effects of limited main memory bandwidth on the strided GHB prefetcher. The system with limited main memory bandwidth ("limited BW") is identical to the system with full BW except that the main memory bandwidth is 800 MT/s, a sixth of the full BW.

Results

The performance of the strided GHB prefetcher in a memory bandwidth limited system is plotted in fig. 3. There are a few observations to make.

- First, the IPC speedups measured in the limited BW system are strictly lower than those from the unconstrained full BW system.
- Next, the IPC speedup on the `charlie_3` workload is below 1, indicating a performance decrease. It achieves the lowest IPC speedup of all workloads. This is most likely due to the combinations of large PPKI, low accuracy and limited bandwidth availability. The largest IPC speedup is observed in the `bfs-14` workload.
- Finally, the largest IPC speedup decreases are on the sssp-10 and sssp-14 traces with `-0.104` and `-0.106` respectively. The smallest IPC speedup decrease is on the `cc-13` workload with `-0.002`.

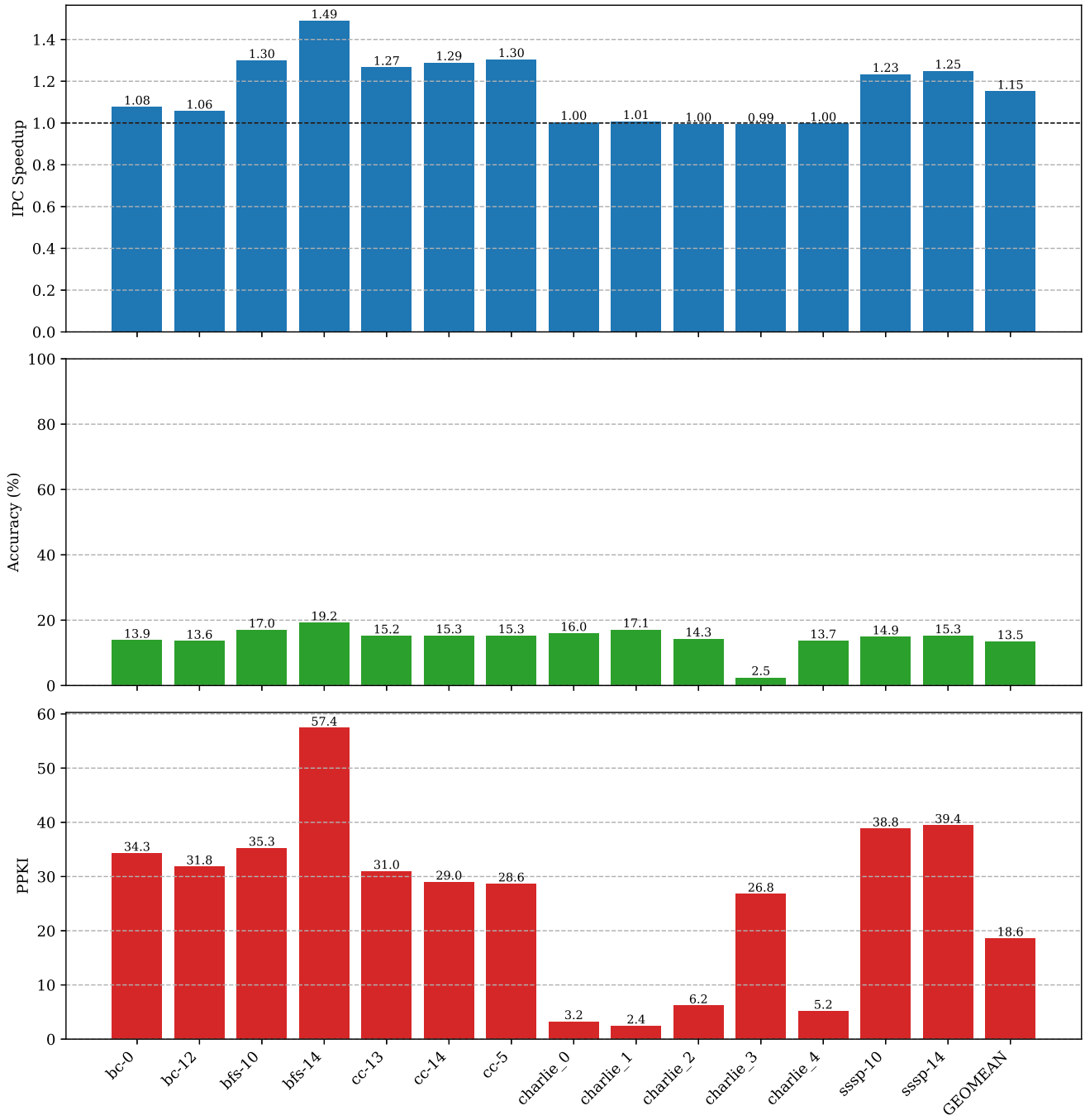

Figure 3: Task 2: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher (1 CPU, limited BW)

Task 3/4: System-Aware Prefetcher Design

As suggested in the task description, the strided GHB prefetcher performs worse in the bandwidth limited system due to the higher cost of speculative prefetches wasting bandwidth. Task 3 therefore is concerned with implementing system feedback, so that the prefetcher throttles its aggressiveness based on a heuristic scheme as laid out in the task description.

Results

The performance metrics of the system aware strided GHB prefetcher in the full BW system and limited BW system are plotted in fig. 4 and fig. 5 respectively.

Full Bandwidth System The bar graph for the full BW system shows that the system-aware prefetcher's accuracy improves on almost all workloads. The IPC speedup values are identical to those of the system-unaware prefetcher.

Limited Bandwidth System The bar graph for the limited BW system shows notably decreased IPC speedups for the system-aware GHB prefetcher on the `bfs-10` and `bfs-14` workloads. Otherwise the IPC speedups either increase, or remain constant. The geometric mean of the IPC speedup decreases by a lower amount than the system-unaware prefetcher in the limited BW environent.

The prefetching accuracy jumps by over `18%` in the geometric mean, because the prefetcher is more careful about which prefetches are issued. This is also reflected upon in the PPKI bar chart, with much fewer prefetches issued by the system-aware prefetcher.

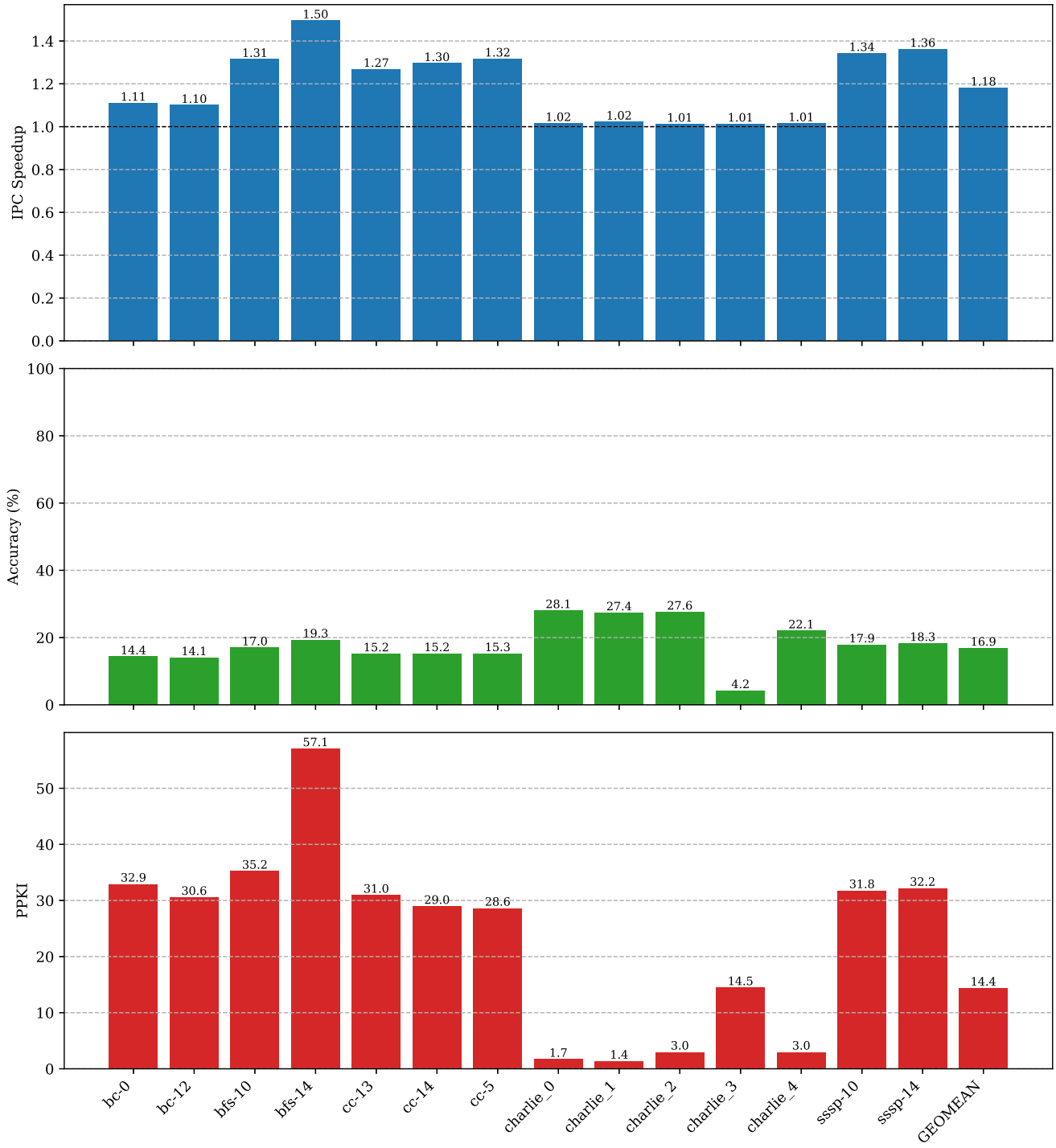

Figure 4: Task 3: System-Aware Strided GHB Prefetcher (1 CPU, *Full* BW)

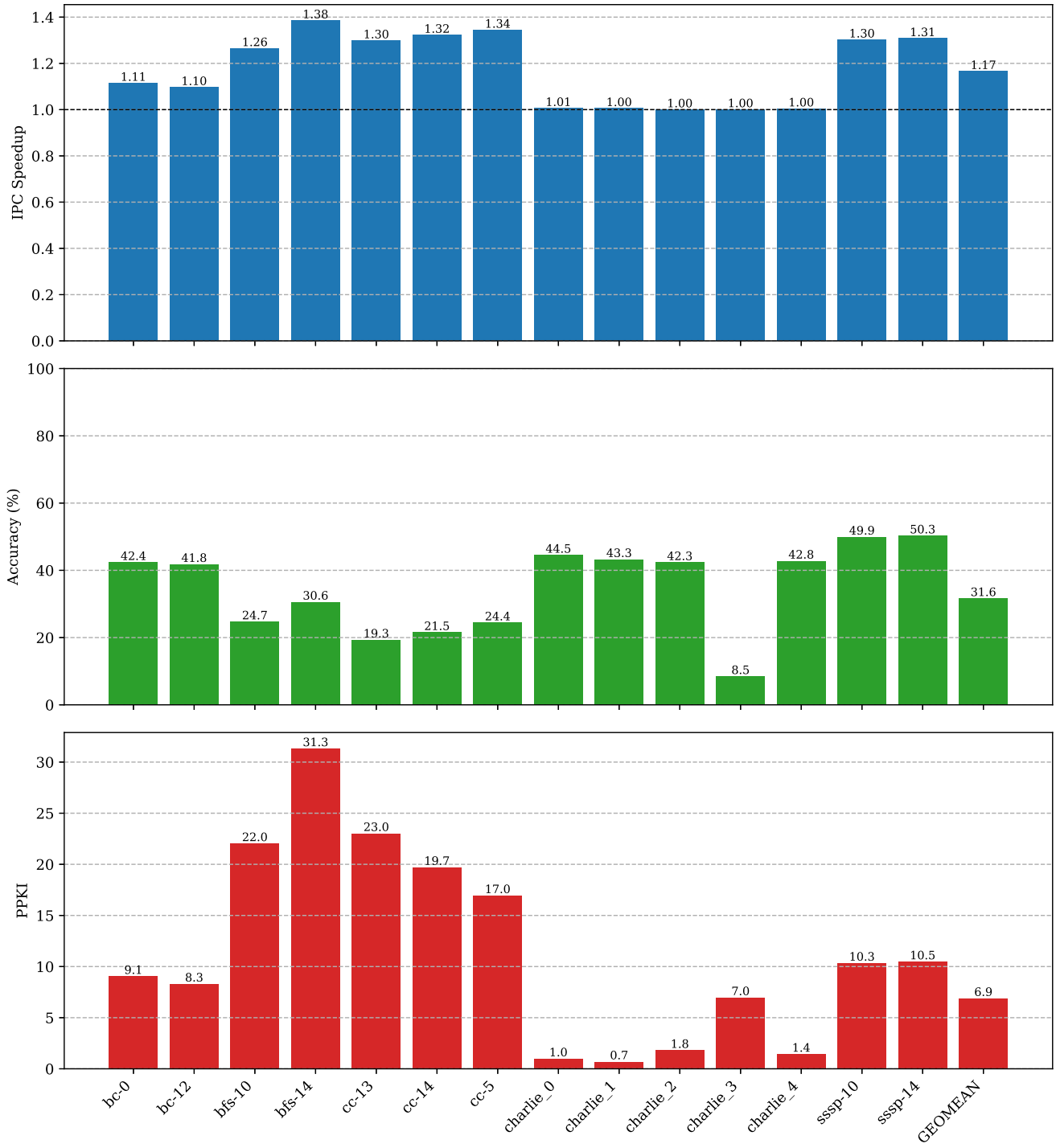

Figure 5: Task 3: System-Aware Strided GHB Prefetcher (1 CPU, *Limited* BW)

Task 4/4: Design Your Own Prefetcher

TBD

Bonus Task: Comparison Against a State-Of-The-Art Prefetcher

In the bonus task, I compare Pythia (Bera et al. 2021), a state-of-the-art prefetcher against the other prefetchers I designed in this lab.

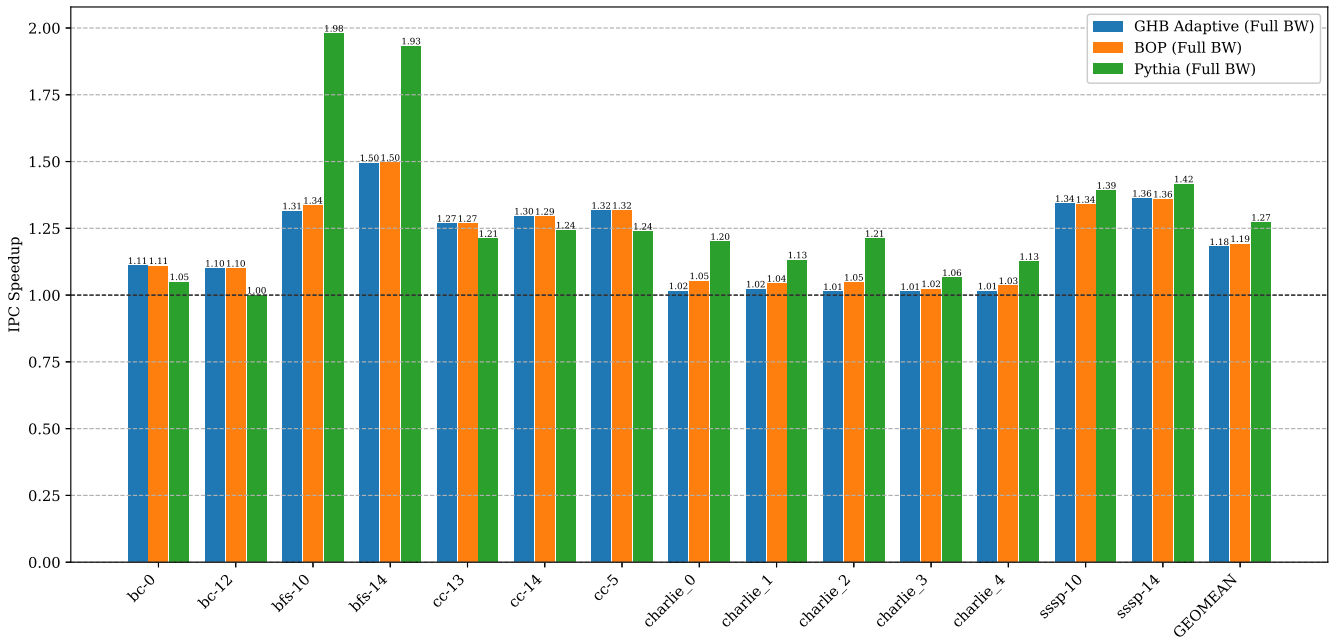

Figure 6: Bonus Task: IPC Speedup Comparison (1 CPU, *full* BW)

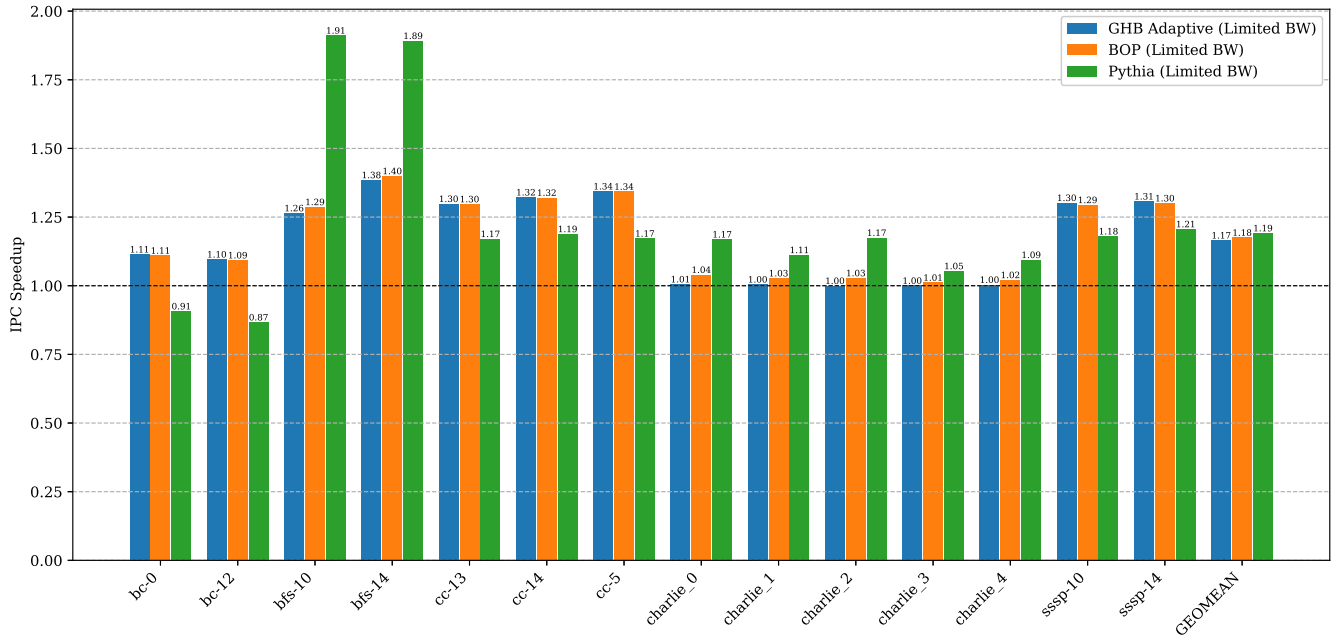

Figure 7: Bonus Task: IPC Speedup Comparison (1 CPU, *Limited* BW)

Citations

Bera, Rahul, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. "Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning." *CoRR* abs/2109.12021. https://arxiv.org/abs/2109.12021.

Gober, Nathan, Gino Chacon, Lei Wang, et al. 2022. *The Championship Simulator: Architectural Simulation for Education and Competition.* https://arxiv.org/abs/2210.14324.

Michaud, Pierre. 2016. "Best-Offset Hardware Prefetching." *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 469–80. https://doi.org/10.1109/HPCA.2016.7446087.

Nesbit, K. J., and J. E. Smith. 2004. "Data Cache Prefetching Using a Global History Buffer." *10th International Symposium on High Performance Computer Architecture (HPCA'04)*, 96–96. https://doi.org/10.1109/HPCA.2004.10030.